VerifyID — Automatic KYC Verification System

Complete Installation & User Manual

Table of Contents

1. System Overview
2. Prerequisites
3. Installation Guide
4. How the System Works
5. User Roles & Permissions
6. API Reference
7. Production Deployment
8. Troubleshooting

System Overview

VerifyID is a full-stack KYC (Know Your Customer) verification system that automates identity verification using OCR (Optical Character Recognition) and facial recognition technologies. The system helps organizations verify customer identities efficiently through:

- **Automated document verification** using OCR technology
- **Facial recognition matching** between selfies and ID documents
- **Role-based access control** for different user types
- **Comprehensive audit logging** for compliance
- **Alert system** for suspicious activities

Prerequisites

System Requirements

Backend Requirements:

- Python 3.11 or higher
- pip package manager
- SQLite3 (development) / PostgreSQL (production)
- Node.js 16+ and npm (for frontend)

Hardware Recommendations:

- Minimum 4GB RAM
- 10GB free disk space
- Internet connection for package installation

Required Libraries

Backend:

```text
Django==5.0
djangorestframework==3.15.0
djangorestframework-simplejwt==5.3.0
django-cors-headers==4.3.1
Pillow==10.2.0
psycopg2-binary==2.9.9  # For PostgreSQL (optional)
```

Frontend:

- React 18
- React Router DOM 6
- Axios
- Vite 5
- Bootstrap Icons

Installation Guide

Step 1: Clone or Download the Project

```bash
# If using git
git clone <repository-url>
cd kyc_system

# Or download and extract the project files
```

Step 2: Backend Setup

2.1 Create Virtual Environment

```bash
cd backend

# On macOS/Linux
python3 -m venv venv
source venv/bin/activate

# On Windows
python -m venv venv
venv\Scripts\activate
```

2.2 Install Python Dependencies

```bash
pip install -r requirements.txt
```

requirements.txt content:

```text
Django==5.0
djangorestframework==3.15.0
djangorestframework-simplejwt==5.3.0
django-cors-headers==4.3.1
Pillow==10.2.0
psycopg2-binary==2.9.9
python-dotenv==1.0.0
pytesseract==0.3.10
face-recognition==1.3.0
```

2.3 Configure Database

```bash
# Run migrations
python manage.py makemigrations
python manage.py migrate

# Create superuser (admin account)
python manage.py createsuperuser
# Follow prompts: username, email, password
```

2.4 Start Backend Server

```bash
python manage.py runserver
# Server runs at http://localhost:8000
```

Step 3: Frontend Setup

3.1 Install Node Dependencies

```bash
cd ../frontend

# Install npm packages
npm install
```

package.json dependencies:

```json
{
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-router-dom": "^6.20.0",
    "axios": "^1.6.0"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.2.0",
    "vite": "^5.0.0"
  }
}
```

3.2 Configure API Proxy

Create vite.config.js:

```javascript
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: 'http://localhost:8000',
        changeOrigin: true
      },
      '/media': {
        target: 'http://localhost:8000',
        changeOrigin: true
      }
    }
  }
})
```

3.3 Start Frontend Development Server

```bash
npm run dev
# Frontend runs at http://localhost:5173
```

Step 4: Verify Installation

1. Open browser to http://localhost:5173
2. You should see the login page
3. Login with superuser credentials created earlier
4. You should be redirected to the dashboard

How the System Works

System Architecture

```text
┌──────────────────────────────────────────────────────────────────────────────┐
│                User Browser                    │
│               (React Frontend)                 │
└──────────────────────────┬──────────────────────────────────────────────────┘
                           │ HTTP/HTTPS
                           ↓
┌──────────────────────────────────────────────────────────────────────────────┐
│             Django Backend (API)               │
│  ┌──────────────────────────────────────────────────────────────────┐  │
│  │ Authentication Layer (JWT)                  │  │
│  ├──────────────────────────────────────────────────────────────────┤  │
│  │ Business Logic Layer                        │  │
│  │ - KYC Processing                            │  │
│  │ - OCR Integration                           │  │
│  │ - Face Recognition                          │  │
│  ├──────────────────────────────────────────────────────────────────┤  │
```

```
│  │ Database Layer                    │  │
│  │ - User Management                 │  │
│  │ - KYC Applications                │  │
│  │ - Audit Logs                      │  │
│  └──────────────────────────────────┘  │
└────────────────────────────────────────┘
```

Core Workflow

1. User Registration & Authentication

- User registers with email and password
- System creates user account with 'customer' role
- JWT tokens generated for authentication
- Tokens included in all subsequent API requests

2. KYC Application Process

Step 1: Submit Personal Information

```text
User → Fills KYC Form → System validates data → Creates application record
```

Step 2: Document Upload

```text
User uploads:
- ID document (front)
- ID document (back)
- Selfie photo

System stores files with unique names
```

Step 3: Automated Verification

```text
System triggers:
1. OCR Processing
   ├─ Extracts text from ID document
```

```
  ├─ Validates document format
  └─ Extracts: Name, ID number, DOB, Address


2. Face Recognition
  ├─ Compares selfie with ID document photo
  ├─ Calculates similarity score
  └─ Determines match threshold (75%+)


3. Risk Assessment
  ├─ Checks for duplicates
  ├─ Validates document authenticity
  └─ Generates risk score
```

Step 4: Review (if needed)

```text
If verification passes → Auto-approve
If verification fails → Flag for manual review
Compliance officer reviews and decides
```

3. OCR Technology Integration

The system uses Tesseract OCR engine for text extraction:

```python
# How OCR works (simplified)
def process_document(image):
    # 1. Preprocess image (enhance contrast, denoise)
    processed_img = preprocess_image(image)

    # 2. Extract text
    extracted_text = pytesseract.image_to_string(processed_img)

    # 3. Parse structured data
    parsed_data = {
        'full_name': extract_name(extracted_text),
        'id_number': extract_id_number(extracted_text),
        'date_of_birth': extract_dob(extracted_text),
        'address': extract_address(extracted_text)
    }
```

```
    return parsed_data
```

4. Facial Recognition System

Uses face_recognition library (based on dlib):

```python
def verify_face_match(selfie, id_photo):
    # 1. Detect faces in both images
    selfie_face = detect_face(selfie)
    id_face = detect_face(id_photo)

    # 2. Generate facial encodings (128-dimension vector)
    selfie_encoding = encode_face(selfie_face)
    id_encoding = encode_face(id_face)

    # 3. Calculate similarity distance
    distance = face_distance(selfie_encoding, id_encoding)

    # 4. Convert to similarity score (0-1)
    similarity_score = 1 - distance

    # 5. Determine match (threshold 0.75)
    is_match = similarity_score >= 0.75

    return similarity_score, is_match
```

Data Flow Diagram

```text
┌──────────┐    ┌──────────┐    ┌──────────┐
│ User     │──▶ │ React    │──▶ │ Django   │
│ Browser  │◀── │ Frontend │◀── │ API      │
└──────────┘    └──────────┘    └──────────┘
                     │
                     ▼
              ┌──────────────────┐
              │ Database         │
```

```
│  ┌──────────────┐        │
│  │ Users    │   │
│  ├──────────────┤   │
│  │ KYC Apps │   │
│  ├──────────────┤   │
│  │ Logs     │   │
│  └──────────────┘   │
└──────────────────────┘
      │
      ▼
┌──────────────────────┐
│ External APIs   │
│ - OCR Service   │
│ - Face Match    │
└──────────────────────┘
```

User Roles & Permissions

Role Hierarchy

text

```
┌──────────────────────────────────────────────┐
│          Admin                  │
│  - Full system access              │
│  - User management                 │
│  - System configuration            │
└──────────────────────┬──────────────────────────┘
           │
┌──────────────────────▼──────────────────────────┐
│     Compliance Officer            │
│  - Review KYC applications          │
│  - Manage alerts                   │
│  - View audit logs                 │
└──────────────────────┬──────────────────────────┘
           │
┌──────────────────────▼──────────────────────────┐
│        Customer                  │
```

```
│  - Submit KYC application          │
│  - View own profile               │
│  - Track application status       │
└──────────────────────────────────────────────────────┘
```

API Reference

Authentication Endpoints

Register User

```http
POST /api/v1/auth/register/
```

Request:

```json
{
  "email": "user@example.com",
  "password": "securepass123",
  "full_name": "John Doe",
  "phone": "+1234567890"
}
```

Response:

```json
{
  "user": {
    "id": 1,
    "email": "user@example.com",
    "full_name": "John Doe",
    "role": "customer"
  },
  "access": "eyJhbGciOiJIUzI1NiIs...",
```

```
  "refresh": "eyJhbGciOiJIUzI1NiIs..."
}
```

Login

```http
POST /api/v1/auth/login/
```

Request:

```json
{
  "email": "user@example.com",
  "password": "securepass123"
}
```

Response: Same as register

KYC Application Endpoints

Create Application

```http
POST /api/v1/kyc-applications/
```

Request:

```json
{
  "full_name": "John Doe",
  "date_of_birth": "1990-01-01",
  "address": "123 Main St",
  "id_type": "passport",
  "id_number": "AB123456"
}
```

Upload Documents

```http
```

```
POST /api/v1/kyc-applications/{id}/upload-documents/
```

Request: multipart/form-data

- id_document_front: file
- id_document_back: file
- selfie: file

Submit for Verification

```http
POST /api/v1/kyc-applications/{id}/submit/
```

Response:

```json
{
  "status": "processing",
  "message": "Application submitted for verification",
  "application_id": 1
}
```

Run OCR

```http
POST /api/v1/kyc-applications/{id}/run-ocr/
```

Response:

```json
{
  "extracted_data": {
    "full_name": "John Doe",
    "id_number": "AB123456",
    "date_of_birth": "1990-01-01",
    "address": "123 Main St"
  },
  "confidence_score": 0.95
}
```

Run Face Match

```http
POST /api/v1/kyc-applications/{id}/run-face-match/
```

Response:

```json
{
  "match_score": 0.87,
  "is_match": true,
  "verification_status": "passed"
}
```

Dashboard Endpoints

Get Statistics

```http
GET /api/v1/dashboard/stats/
```

Response:

```json
{
  "total_applications": 150,
  "pending_review": 23,
  "approved_today": 12,
  "rejected_today": 3,
  "average_processing_time": 45,
  "alerts_count": 5
}
```

Production Deployment

Backend Production Setup

1. Use PostgreSQL Database

```python
# settings.py
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': os.environ.get('DB_NAME'),
        'USER': os.environ.get('DB_USER'),
        'PASSWORD': os.environ.get('DB_PASSWORD'),
        'HOST': os.environ.get('DB_HOST'),
        'PORT': os.environ.get('DB_PORT', '5432'),
    }
}
```

2. Configure Environment Variables

Create .env file:

```env
DEBUG=False
SECRET_KEY=your-secret-key-here
ALLOWED_HOSTS=yourdomain.com,api.yourdomain.com
DB_NAME=kyc_db
DB_USER=kyc_user
DB_PASSWORD=secure_password
DB_HOST=localhost
```

3. Static & Media Files

```python
# settings.py
STATIC_URL = '/static/'
```

```
STATIC_ROOT = '/var/www/kyc/static/'
MEDIA_URL = '/media/'
MEDIA_ROOT = '/var/www/kyc/media/'
```

4. Gunicorn Setup

```bash
pip install gunicorn
gunicorn --workers 4 --bind 0.0.0.0:8000 kyc_project.wsgi:application
```

5. Nginx Configuration

```nginx
server {
    listen 80;
    server_name yourdomain.com;

    location / {
        proxy_pass http://localhost:5173;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    location /api/ {
        proxy_pass http://localhost:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    location /media/ {
        alias /var/www/kyc/media/;
    }
}
```

Frontend Production Build

```bash
```

```
cd frontend
npm run build
# Build output in 'dist' folder
```

Serve with Nginx:

```nginx
location / {
    root /var/www/kyc/frontend/dist;
    try_files $uri $uri/ /index.html;
}
```

Troubleshooting

Common Issues & Solutions

1. CORS Errors

Error: Access to fetch at 'http://localhost:8000/api/...' from origin 'http://localhost:5173' has been blocked by CORS policy

Solution:

```python
# settings.py
CORS_ALLOWED_ORIGINS = [
    "http://localhost:5173",
    "http://127.0.0.1:5173",
]
CORS_ALLOW_CREDENTIALS = True
```

2. Database Migration Issues

```bash
# Reset migrations (development only)
python manage.py migrate --fake kyc_app zero
```

```
python manage.py makemigrations
python manage.py migrate
```

3. File Upload Size Limit

Error: Request body exceeded settings.DATA_UPLOAD_MAX_MEMORY_SIZE

Solution:

```python
# settings.py
DATA_UPLOAD_MAX_MEMORY_SIZE = 10485760  # 10MB
```

4. JWT Token Expired

Error: Token is invalid or expired

Solution:

```javascript
// Frontend: Implement token refresh
api.interceptors.response.use(
  response => response,
  async error => {
    if (error.response.status === 401) {
      // Refresh token
      const refresh = localStorage.getItem('refresh');
      const response = await api.post('/auth/token/refresh/', { refresh });
      localStorage.setItem('access', response.data.access);
      // Retry original request
    }
    return Promise.reject(error);
  }
);
```

Performance Optimization

1. Database Indexing

```python
```

```python
# models.py
class KYCApplication(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE, db_index=True)
    status = models.CharField(max_length=20, db_index=True)
    created_at = models.DateTimeField(auto_now_add=True, db_index=True)
```

2. Caching

```python
# settings.py
CACHES = {
    'default': {
        'BACKEND': 'django.core.cache.backends.redis.RedisCache',
        'LOCATION': 'redis://127.0.0.1:6379/1',
    }
}
```

3. API Pagination

```python
# views.py
class KYCApplicationViewSet(viewsets.ModelViewSet):
    pagination_class = PageNumberPagination
    pagination_class.page_size = 20
```

Security Best Practices

1. **Always use HTTPS in production**
2. **Store secrets in environment variables**
3. **Implement rate limiting**

```python
# Install django-ratelimit
pip install django-ratelimit

# Apply to views
@ratelimit(key='ip', rate='5/m')
def login(request):
    # Rate limited to 5 requests per minute per IP
```

4. **Sanitize file uploads**

```python
ALLOWED_FILE_TYPES = ['image/jpeg', 'image/png', 'image/jpg']
MAX_FILE_SIZE = 5 * 1024 * 1024  # 5MB
```

5. **Regular security updates**

```bash
pip install safety
safety check -r requirements.txt
```

Support & Maintenance

Monitoring

Check application logs:

```bash
# Backend logs
tail -f /var/log/kyc/backend.log

# Nginx access logs
tail -f /var/log/nginx/access.log

# Nginx error logs
tail -f /var/log/nginx/error.log
```

Backup Strategy

```bash
# Database backup (PostgreSQL)
pg_dump kyc_db > backup_$(date +%Y%m%d).sql
```

```
# Media files backup
tar -czf media_backup_$(date +%Y%m%d).tar.gz /var/www/kyc/media/
```

Regular Maintenance Tasks

1. **Weekly**: Check disk space, review error logs
2. **Monthly**: Update dependencies, review security patches
3. **Quarterly**: Test backup restoration, review access logs

Additional Resources

- **Django Documentation**: https://docs.djangoproject.com/
- **DRF Documentation**: https://www.django-rest-framework.org/
- **React Documentation**: https://reactjs.org/docs/
- **Tesseract OCR**: https://github.com/tesseract-ocr/tesseract
- **Face Recognition Library**: https://github.com/ageitgey/face_recognition

Conclusion

The VerifyID system provides a comprehensive KYC verification solution with automated document processing and facial recognition capabilities. By following this guide, you can successfully install, configure, and deploy the system in both development and production environments.